
Learning Letters and Words in ASL Using Keypoint Sequences

Madelyn Redick
Northeastern University
Boston, MA

Nicole Ni
Northeastern University
Boston, MA

Dhruv Jatkar
Northeastern University
Boston, MA

1 Background and Motivation

We propose to develop a lightweight American Sign Language (ASL) captioning system that combines the recognition of finger-spelled letters with a limited vocabulary of high-frequency signs. The project will be structured in two stages. Stage 1 will strain a robust letter recognizer using labeled A-Z fingerspelling video clips. Stage 2 will extend the system to a small lexicon of approximately 100-200 common ASL signs (e.g., hello, thank you, please, help, water, food), leveraging publicly available video sources such as ASL club recordings. Hand keypoints will be extracted from video frames using pose estimation and assembled into temporal sequences, which will then be classified with a lightweight temporal model. For sentence-level demonstrations, simple motion-based segmentation (there are several ways to do this, e.g. [4]) will divide continuous video into sign units; decoding will prioritize word-level predictions while falling back on letters in cases of uncertainty. This modular design allows the system to produce intelligible captions without reliance on large datasets or complex grammar models. As an optional extension, this project will leverage the substantial progress in natural language processing (NLP) achieved over the past five years. While the core system focuses on producing caption-like outputs (letters and high-frequency words) directly from video input, the integration of modern NLP method (such as a Large Language Model) opens the possibility of moving from surface-level captioning toward meaning-preserving translation. Specifically, the outputs of the vision-based classifier—letters, words, or glossed signs—can be fed into an English language model trained on large text corpora. This secondary model would serve to contextualize partial or noisy predictions, reorder sequences into more natural English syntax, and, in certain cases, infer intended meaning beyond the literal sign-by-sign output.

The main contribution of this project lies in context-aware decoding. Rather than treating each recognized segment independently, the system will integrate a simple language model that exploits patterns in ASL usage, such as the high frequency of certain collocations (thank you → please, yes/no → handshake/nod). By constraining decoding with these usage patterns, the system can improve robustness and readability even when the individual sign recognition is uncertain. This approach remains lightweight by using n-gram models rather than a full translational pipeline, but adds an underexplored linguistic layer that enhances *practical* usability.

2 Literary Survey

In Sign Language Recognition with Transformer Networks, De Coster et al. introduce four methods for sign language recognition [1], in which we focus on the two transformer-based architectures: the Pose Transformer Network (PTN) and the Video Transformer Network (VTN). Each is tailored to different inputs. The PTN leverages 2D keypoints extracted via OpenPose. In contrast, the VTN processes features that were first extracted using a 2D CNN. The results show that PTN achieves competitive accuracy, but VTN demonstrates superior performance, especially on visually complex signs where keypoint extraction may fail to capture details. However, VTN requires more computational resources and data to generalize effectively. For our project, we will use the PTN architecture for gesture-based sign recognition despite its poorer performance. It allows our system

to be modular and have a fallback system, both extremely beneficial considering our limited time and resources. Future development for our project could explore the use of the VTN architecture for greater performance.

Gong et al. (CVPR 2024) introduces SignLLM, a pipeline that makes sign videos "language-like" so an off-the-shelf, frozen LLM can perform translation. The methodology is as follows [3]. First, the videos are discretized into short, character-level VQ-Sign tokens, and then uses a "Codebook Reconstruction and Alignment" module to compose word-level tokens. Feeding these tokens into a prompted LLM (LLaMa-7B) yields strong sign-to-text performance. This is an example of *not* having two separate transformer modules, which could be useful for us in a discussion section.

Uthus et al. (NeurIPS 2023 D&B) present YouTube-ASL, a large, open-domain ASL-English parallel corpus mined from YouTube [5]. As baselines, the authors project MediaPipe Holistic landmarks into a T5 sequence model for ASL to English and evaluate on a specific benchmark, achieving a new SOTA and reporting the first zero-shot results. Beyond the scale, the dataset's diversity and signer count aim to improve generalization (which they achieved for the first time). This work is a great example of using weakly supervised web data as a good foundation for LLM-assisted pipelines.

Mishra et al. (IASC 2023) present a three-stage pipeline for hand gesture recognition—detection, tracking, and recognition—focusing on robustness across lighting, background, and user variation [2]. Detection uses a combination of skin color modeling and edge detection, while tracking is guided by inter-frame spatial proximity (often using a Kalman Filter). Recognition relies on template matching, comparing candidates against gesture prototypes at the pixel level. This paper is a helpful reference for foundational gesture recognition pipelines, especially when discussing segmentation and spatial-temporal modeling.

3 Potential Datasets

To train the model on ASL letters, the dataset Images of American Sign Language (ASL) Alphabet Gestures from East West University could be useful. It has 5 folders for image types, one being the raw gestures and the rest are overlayed with points and lines representing the gesture. For our purposes, we will only be using the raw gesture images to train the model on what sign language looks like in the real world. The raw gesture folder consists of 28 image classes, 26 for the letters of the alphabet, and two for the special characters "delete" and "space". There are 42000 image files (.jpg) in total with a variety of angles and lighting, and 1500 images for each of the 28 classes. The dataset Sign Language Dataset by Waseim Nagah Hennes can be used to train the model on ASL words. It contains about 12000 videos in total with roughly 6 to 10 videos files (.mp4) for each of the 2000 words included.

4 Plan of Activities

**Model development may be parallelized.*

1. Week 1 (Starting Oct 13): Data Collection

- (a) More research
- (b) Set up environment
- (c) Find datasets of finger-spelling and common sign words
- (d) Clean and annotate data if needed

Deliverable: Collected necessary data

2. Week 2: Finger-spelling Model Implementation

- (a) Pre-process data
 - Normalize data and augment data
 - Class balancing if needed
- (b) Plan CNN model architecture
- (c) Implement initial model

3. Week 3: Finger-spelling Model Evaluation

- (a) Train model for A–Z finger-spelling
- (b) Evaluate model
- (c) Improve model

Deliverable: Working letter recognizer

4. **Week 4: Gesture-Based Sign Model Implementation**

- (a) Pre-process data
 - Extract hand keypoints
 - Normalize and augment data
 - Class balancing if needed
- (b) Plan temporal model architecture (RNN or Transformer)
- (c) Implement initial model

5. **Week 5: Gesture-Based Sign Model Evaluation**

- (a) Train model for gesture-based signs
- (b) Evaluate model
- (c) Improve model

Deliverable: Working recognizer for 100–200 signs

6. **Week 6: Segmentation and Fallback Logic**

- (a) Implement motion-based segmentation
- (b) Evaluate segmentation accuracy
- (c) Implement fallback letter recognition logic

7. **Week 7: System Integration**

- (a) Integrate all parts (letters, words, and segmentation)
- (b) System testing
- (c) Debugging

Deliverable: Working end-to-end system for captioning ASL

8. **Week 8: Enhancements**

- (a) Continue bug fixes and improvements if needed
- (b) Implement with n-gram language model or LLM integration
- (c) Evaluate level of improvement

Deliverable: Enhanced working system for interpreting ASL

9. **Week 9: Finalization**

- (a) Write report
- (b) Collect final evaluation/accuracy stats
- (c) Create presentation slides

For division of work, collaboration will be required on integral parts such as planning model architecture and verifying deliverables at the end of each week/stage. For each model, the work will be roughly divided into preparation, implementation, and improvement. Weeks 1, 6, and 7 can be split equally by subtask. All members will contribute to the final report and presentation.

References

- [1] Mathieu De Coster, Mieke Van Herreweghe, and Joni Dambre. Sign language recognition with transformer networks. In *Proceedings of the 12th International Conference on Language Resources and Evaluation (LREC 2020)*, pages 6018–6024, 2020.
- [2] S Gnanapriya and K Rahimunnisa. A hybrid deep learning model for real time hand gestures recognition. *Intelligent Automation & Soft Computing*, 36(1), 2023.
- [3] Jia Gong, Lin Geng Foo, Yixuan He, Hossein Rahmani, and Jun Liu. Llms are good sign language translators. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18362–18372, 2024.
- [4] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- [5] Dave Uthus, Garrett Tanzer, and Manfred Georg. Youtube-asl: A large-scale, open-domain american sign language-english parallel corpus. *Advances in Neural Information Processing Systems*, 36:29029–29047, 2023.